Introduction to R

Molecular Approaches to Biodiversity and Ecology

Nasta Lianguzova

Faculty of Biology, Shenzhen MSU-BIT University

2026-10-30

About this course

- How to analyse molecular data in R: from a table of numbers to a figure and a conclusion.
- The statistical methods behind most analyses of molecular data: what they do and when to use them.
- If time allows: working directly with sequencing results, e.g. assembling a mitochondrial genome.

Course roadmap

1. R and data visualization: R basics, ggplot2 (today: R basics)
2. Statistical foundations: descriptive statistics, hypothesis testing, linear models, ANOVA, generalized linear models (GLM)
3. Multivariate methods: PCA, nMDS, clustering; PERMANOVA if time allows
4. Molecular applications: differential expression in RNA-seq, enrichment analysis
5. If time allows: genome assembly

How the course works

- Each session: lecture + hands-on practice in R.
- After each session: a Moodle quiz, required, to be completed before the next session, + optional tasks for bonus points.
- AI tools: use them to understand errors and concepts, not to solve tasks for you (details in Moodle).

Today you will be able to

- Set up R and RStudio and organise your course folder as an RStudio project
- Find your way around the RStudio window and get help on functions
- Create variables and vectors and select elements from them
- Work with the main data types: numeric, character, logical, factors
- Deal with missing values and write a simple function of your own
- Store a data table in a data frame and select rows, columns and cells
- Read a table from a CSV file and save one

Getting started with R

Download and install

Install **R** first, then **RStudio**, an **IDE** (integrated development environment) for R.

1. **R**: open the Tsinghua University CRAN mirror (fast from China) https://mirrors.tuna.tsinghua.edu.cn/CRAN/, then `Download R for Windows -> base` and download the `.exe` installer. Run it and accept the default settings.
2. **RStudio**: https://posit.co/download/rstudio-desktop/, then download RStudio Desktop for Windows (`.exe`). Run it and accept the default settings.
3. Start **RStudio** from the Start menu (not R): RStudio finds R by itself.

macOS / Linux: on macOS, download the R `.pkg` for your processor (Apple silicon or Intel) and the RStudio `.dmg`; on Linux, install R from your distribution's packages (e.g. `r-base` on Ubuntu) and the RStudio `.deb`/`.rpm`.

Your course folder

1. Create a folder for ALL course materials, e.g. `moldiveco` in `Documents`. This folder will be your **working directory**. Save ALL your code files (`.R`) here.
2. Inside it, create a folder `data` for all data files.

You should end up with something like this:

```
C:\Users\<your name>\Documents\moldiveco\
C:\Users\<your name>\Documents\moldiveco\data\
```

- Use only Latin letters, numbers and _ in folder and file names: no spaces, no Chinese characters.
- Windows hides file extensions by default. To see `.R` and `.Rproj`, turn them on in File Explorer: Windows 11: `View -> Show -> File name extensions`; Windows 10: `View` tab, tick `File name extensions`.

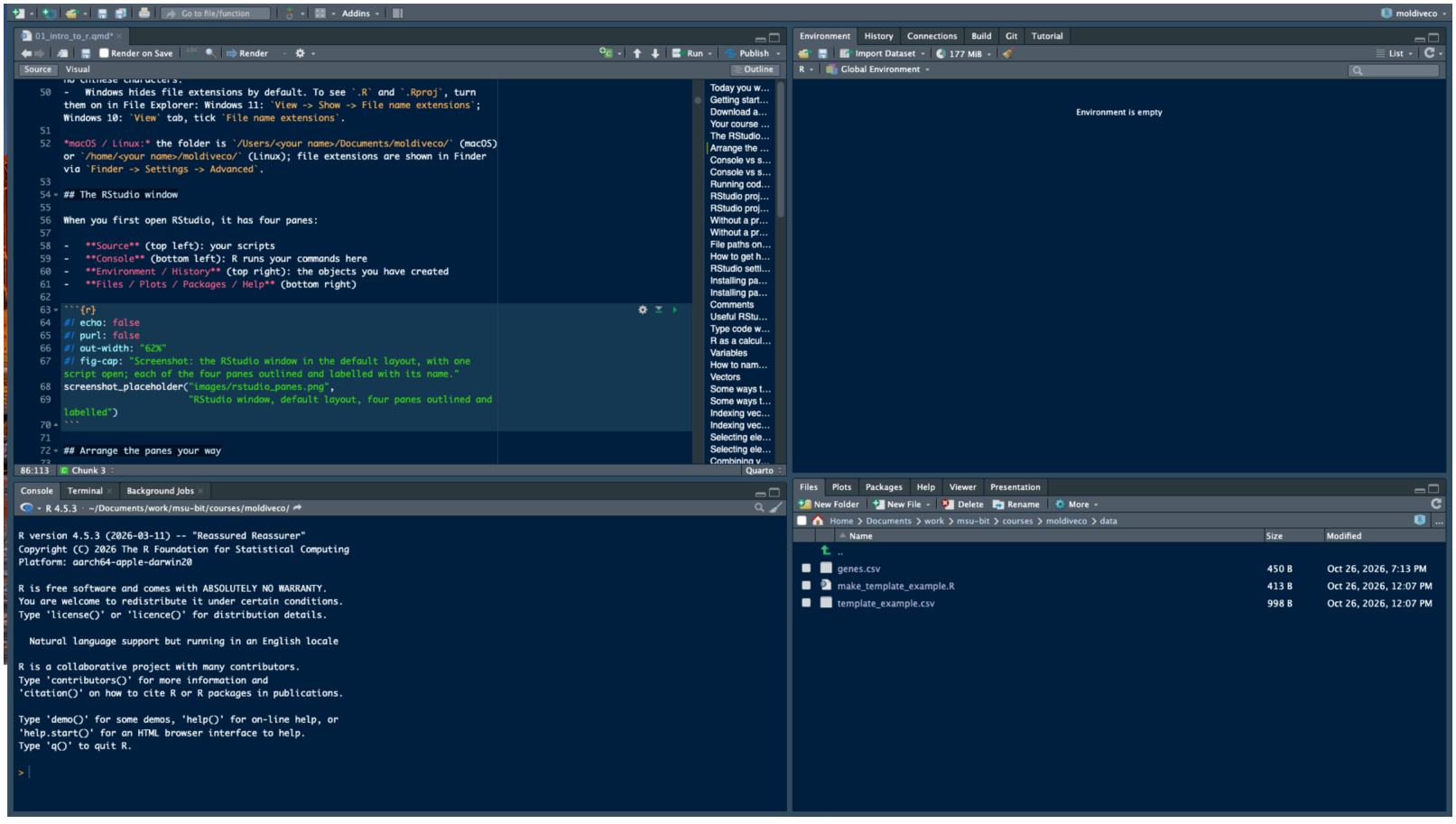
macOS / Linux: the folder is `/Users/<your name>/Documents/moldiveco/` (macOS) or `/home/<your name>/moldiveco/` (Linux); file extensions are shown in Finder via `Finder -> Settings -> Advanced`.

The RStudio window

When you first open RStudio, it has four panes:

- **Source** (top left): your scripts
- **Console** (bottom left): R runs your commands here
- **Environment / History** (top right): the objects you have created
- **Files / Plots / Packages / Help** (bottom right)

The RStudio window: the four panes

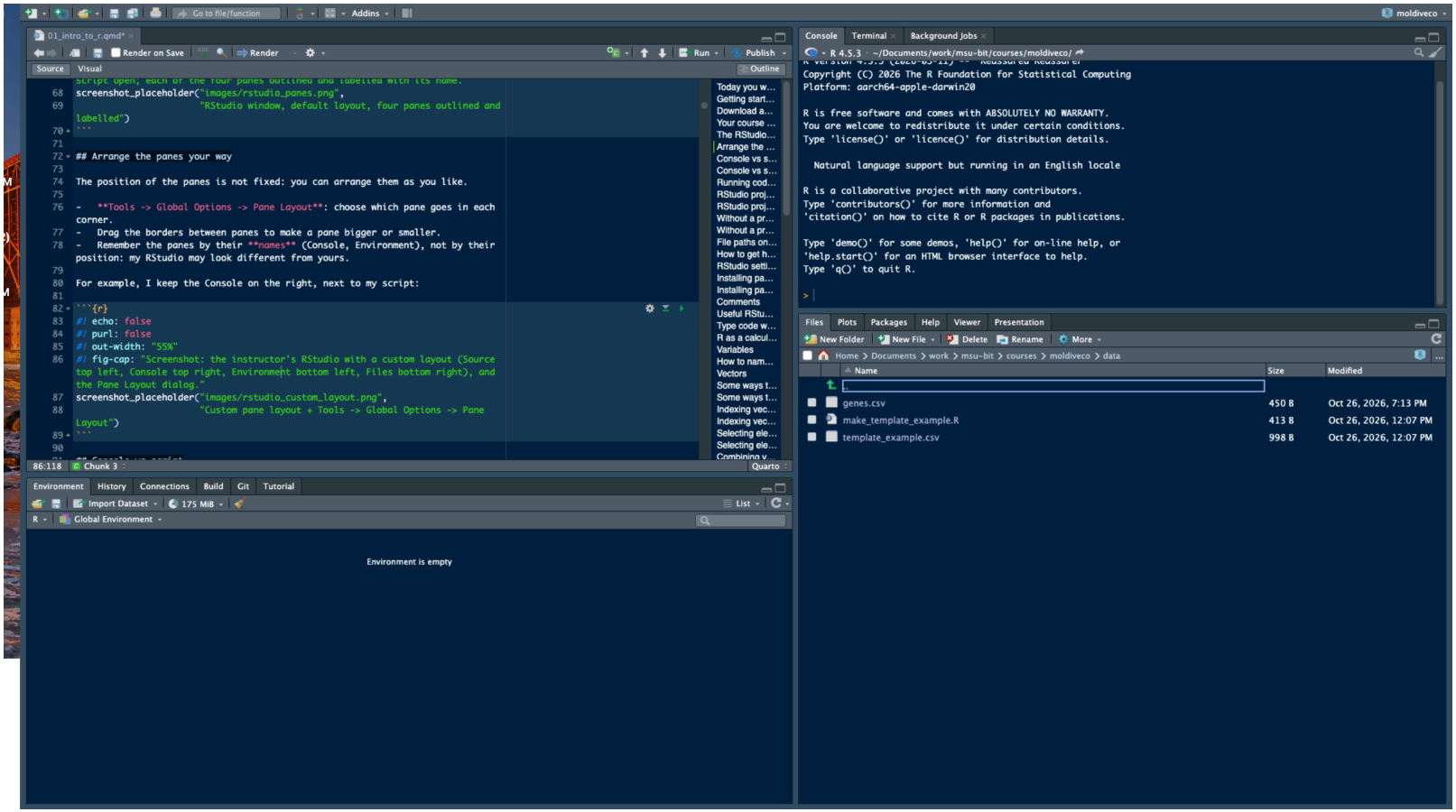


The four panes of the RStudio window.

Arrange the panes your way

- The position of the panes is not fixed: you can arrange them as you like.
- `Tools -> Global Options -> Pane Layout`: choose which pane goes in each corner.
- Drag the borders between panes to make a pane bigger or smaller.
- Remember the panes by their **names** (Console, Environment…), not by their position: my RStudio may look different from yours.
- For example, I keep the Console on the right, next to my script:

Arrange the panes your way: in RStudio



A custom pane layout.

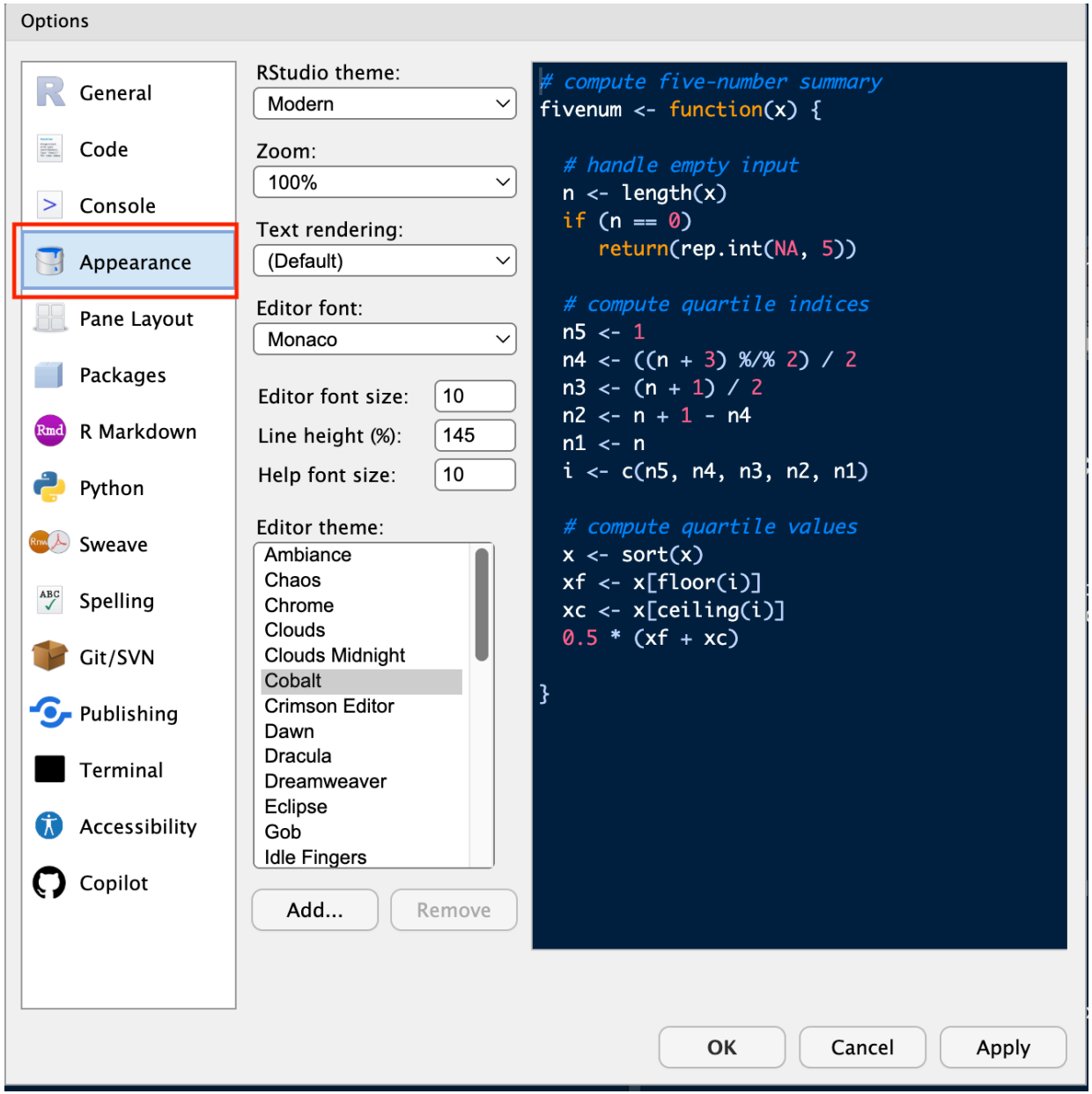
Customize the appearance

`Tools -> Global Options -> Appearance`:

- Editor theme: the colours of the code editor, e.g. a dark theme.
- Editor font size: the size of the code text.
- Zoom: the size of the whole RStudio window.

Changes apply after you click `Apply`. They only change how RStudio looks, not your code.

Customize the appearance: in RStudio



Tools -> Global Options -> Appearance

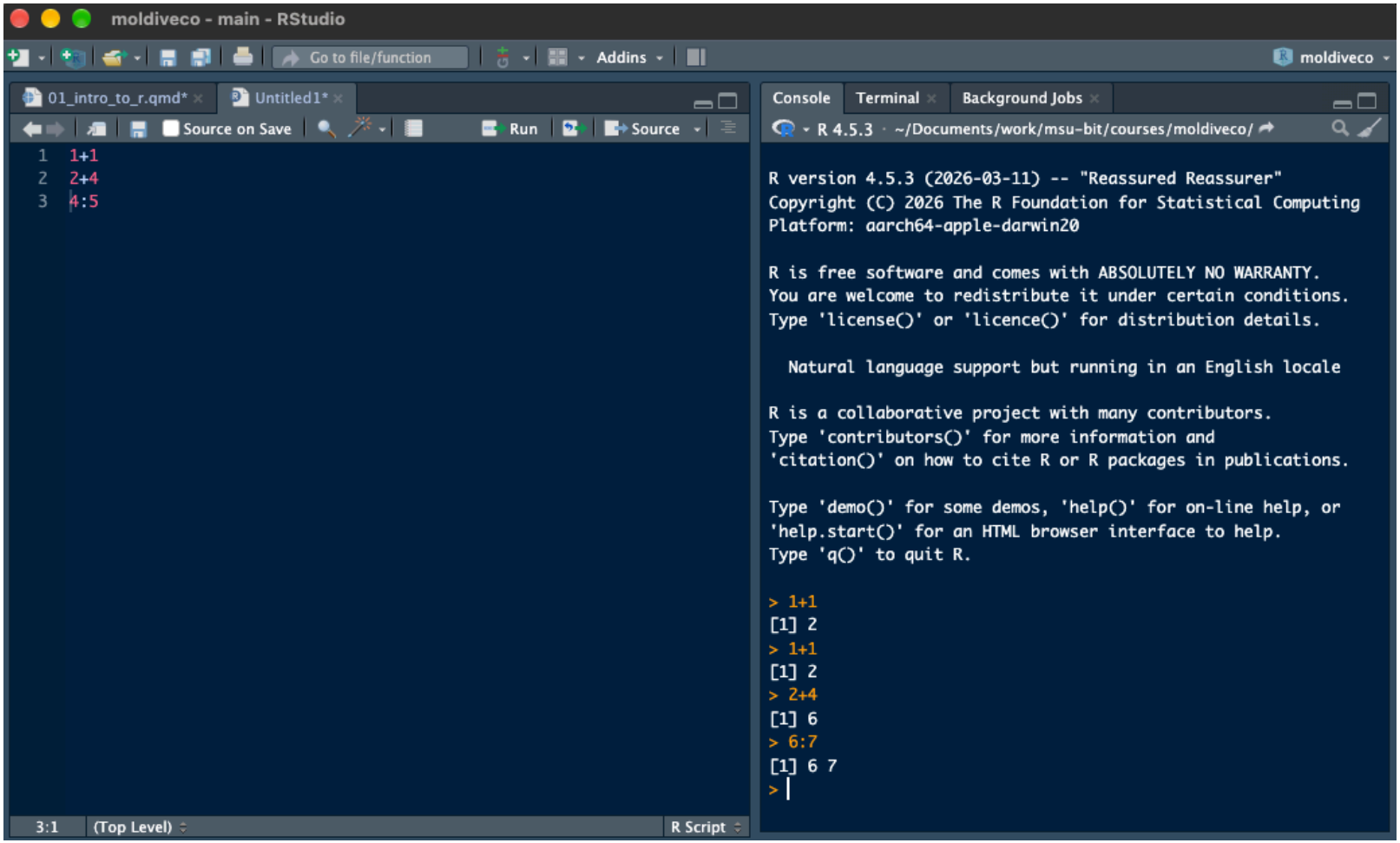
Console vs script

- **Console**: R runs a command as soon as you press `Enter`. Good for quick checks, but nothing is saved.
- **Script**: a text file with your code. Create one with `File -> New File -> R Script` (`Ctrl + Shift + N`). Save it with `Ctrl + S` into your course folder, e.g. `C:\Users\<your name>\Documents\moldiveco\01_intro.R`; then you can run the whole analysis again at any time.

Write all code you want to keep in a script.

macOS: `Cmd` instead of `Ctrl` (`Cmd + Shift + N`, `Cmd + S`).

Console vs script: in RStudio



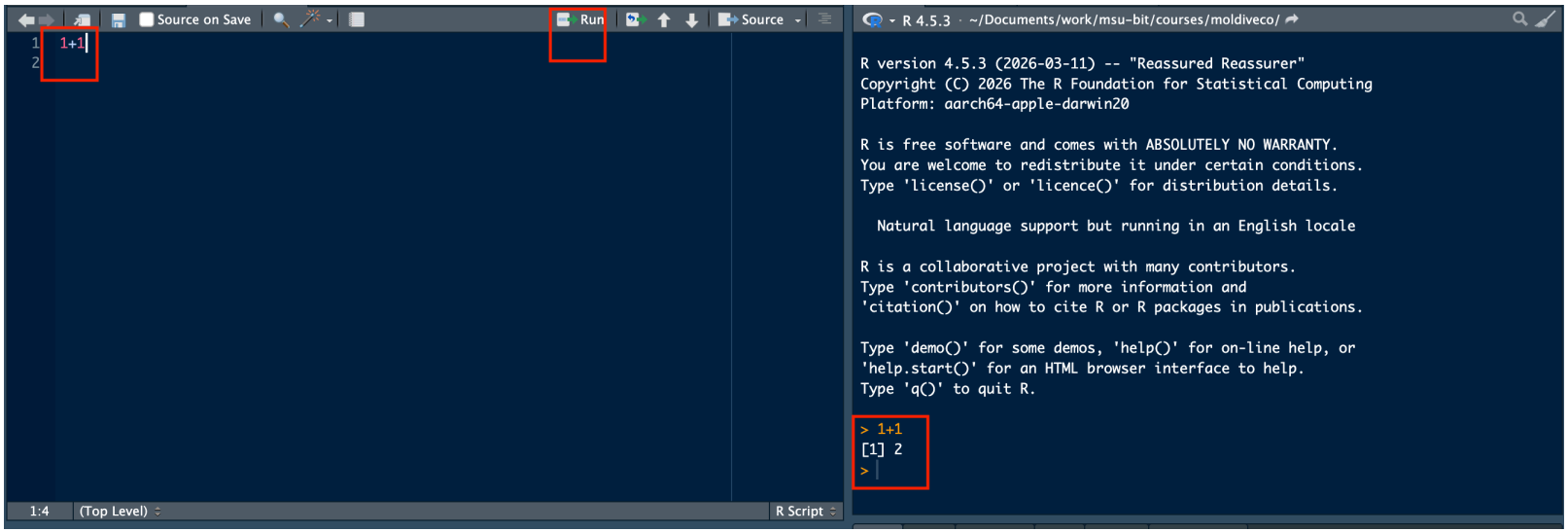
Lines typed in a script (left) and their results in the Console (right).

Running code from a script

- Put the cursor on a line and press `Ctrl + Enter` or click **Run**: the line is sent to the console.
- Select several lines first to run them all.
- `Ctrl + Shift + Enter` runs the whole script.

macOS: `Cmd + Enter` and `Cmd + Shift + Enter`.

Running code from a script: in RStudio



Run sends the current line (1+1) to the Console, which prints the result.

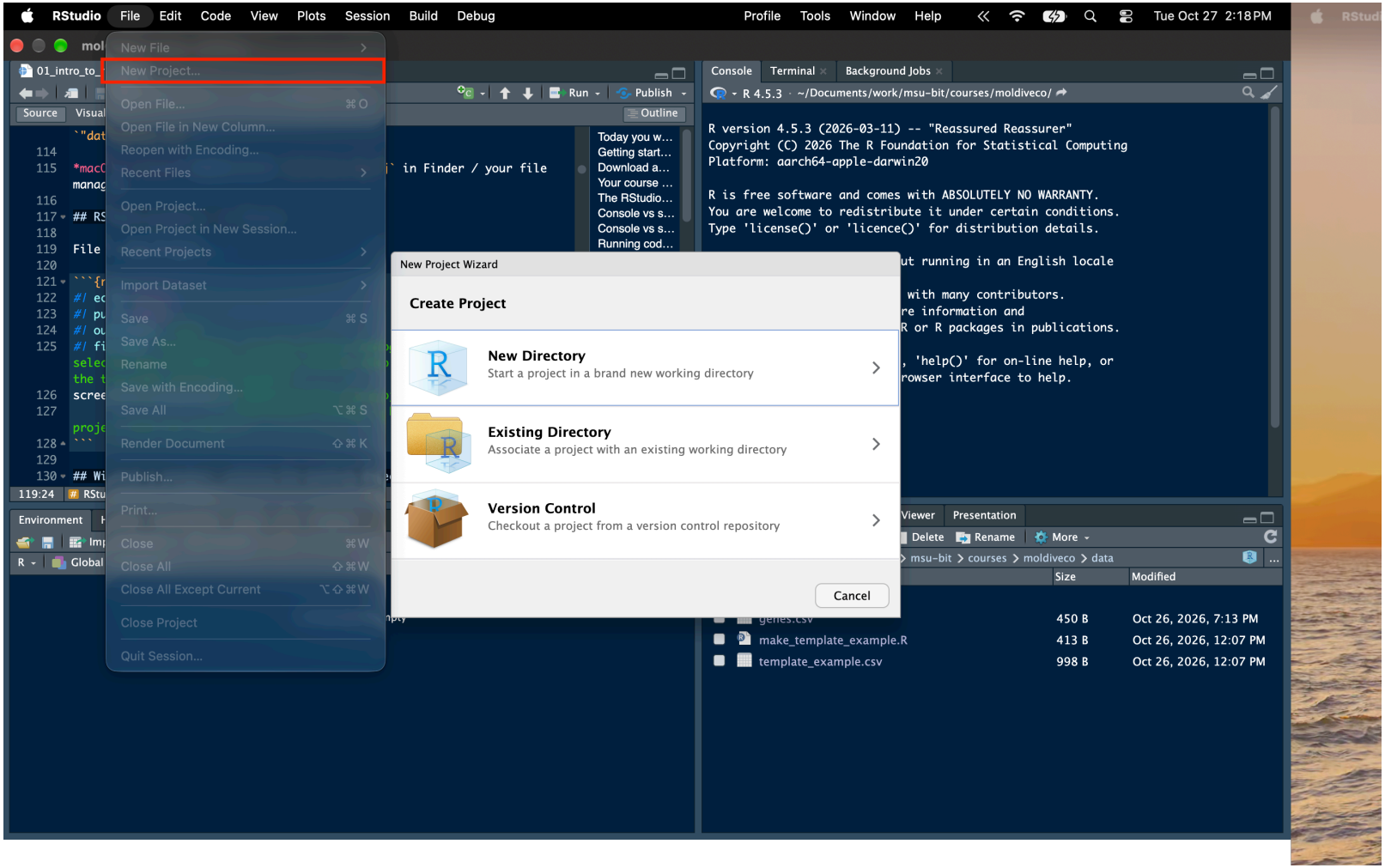
RStudio projects

In this course, always work inside an RStudio **project**: your course folder with a `.Rproj` file in it. RStudio then uses the project folder as the working directory, so you never have to set it by hand.

- Create the project once: `File -> New Project -> Existing Directory`, then choose your `moldiveco` folder.
- Every time you work: double-click `moldiveco.Rproj` in File Explorer. RStudio opens with the working directory already set.
- Files inside the project are then found with short paths.
- Keep all data files in `data/`, images and plots in `images/`, and scripts (`.R`) in the project folder itself. Then paths are always short and predictable, e.g. `"data/genes.csv"`.

macOS / Linux: double-click `moldiveco.Rproj` in Finder / your file manager.

RStudio projects: in RStudio



`File -> New Project...`, then Existing Directory and your `moldiveco` folder.

Without a project: setting the working directory

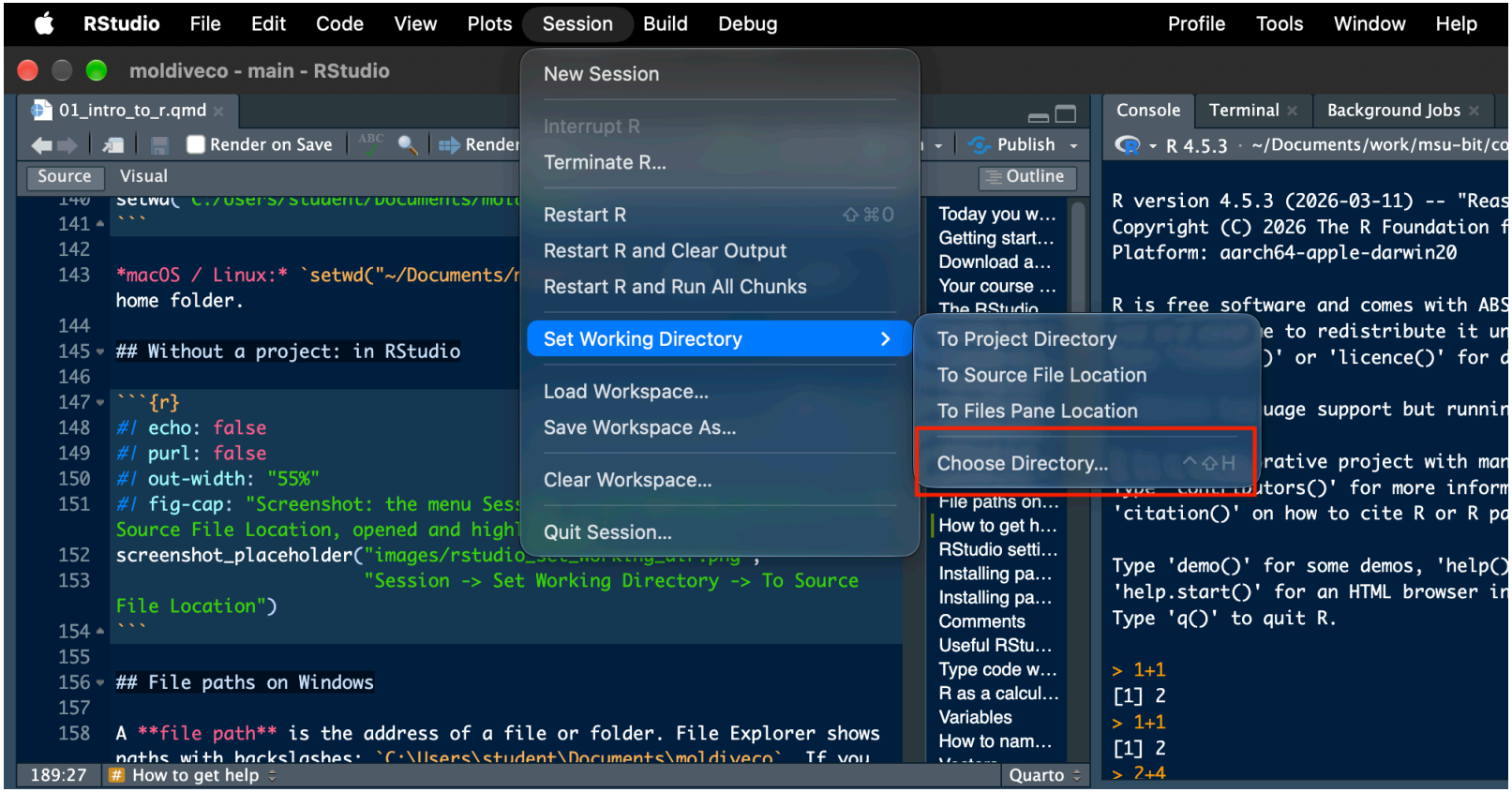
If you open a script without its project, set the working directory yourself:

1. Menu `Session -> Set Working Directory -> To Source File Location` (once your script is saved).
2. Menu `Session -> Set Working Directory -> Choose Directory...`, then pick your course folder.
3. In the `Files` pane, go to the folder with your code, then `More (gear icon) -> Set As Working Directory`.
4. Type the path in quotation marks as the argument of `setwd()` (why `/` and not `\`: see "File paths on Windows"):

```
setwd("C:/Users/student/Documents/moldiveco")
```

macOS / Linux: `setwd("~/Documents/moldiveco")`, where `~` is your home folder.

Without a project: in RStudio



`Session -> Set Working Directory -> Choose Directory...`

File paths on Windows

A **file path** is the address of a file or folder. File Explorer shows paths with backslashes: `C:\Users\student\Documents\moldiveco`. If you paste such a path into R as it is, you get an error, because inside R text `\` starts a special character (e.g. `\n` is a new line):

```
setwd("C:\Users\student\Documents\moldiveco")
# Error: '\U' used without hex digits in character string
```

In R, write Windows paths with `/` (recommended) or with a double backslash `\\`:

```
setwd("C:/Users/student/Documents/moldiveco")
setwd("C:\\Users\\student\\Documents\\moldiveco")
```

Inside an RStudio project you rarely need a full path: use a **relative path**, which starts from the working directory, e.g. `"data/genes.csv"`.

macOS / Linux: paths already use `/`, e.g. `/Users/student/Documents/moldiveco`, so you can paste them as they are.

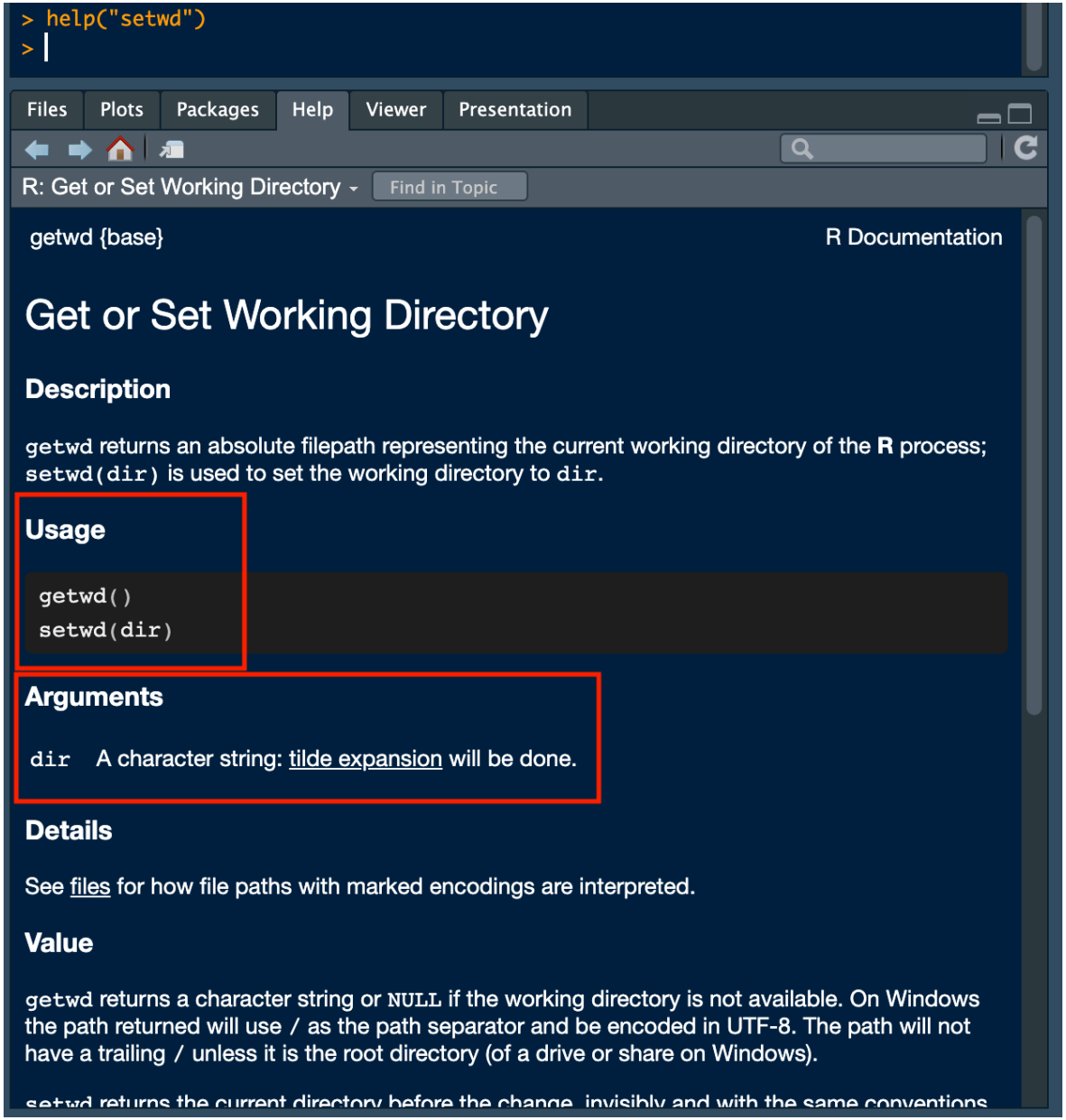
How to get help

1. In RStudio, put the cursor on a function name, e.g. `setwd`, and press `F1`.
2. Type a question mark before the function name and run the line: `?setwd`.
3. Use the function `help()`:

```
help("setwd")
```

macOS: on a laptop keyboard you may need `Fn + F1`.

How to get help: in RStudio



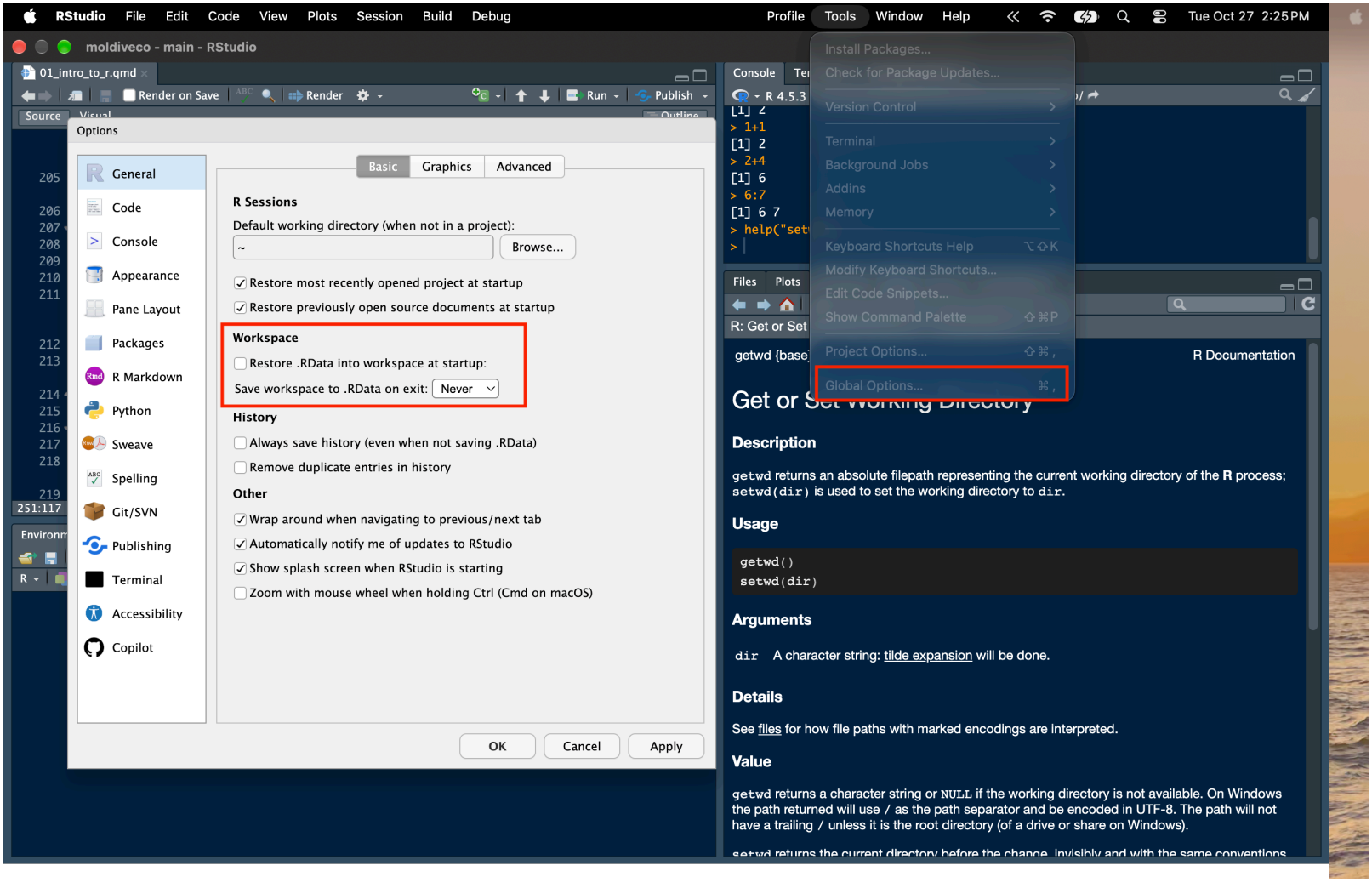
`help("setwd")` opens this page: read Usage and Arguments first.

RStudio settings

All RStudio settings are in the menu `Tools -> Global Options`.

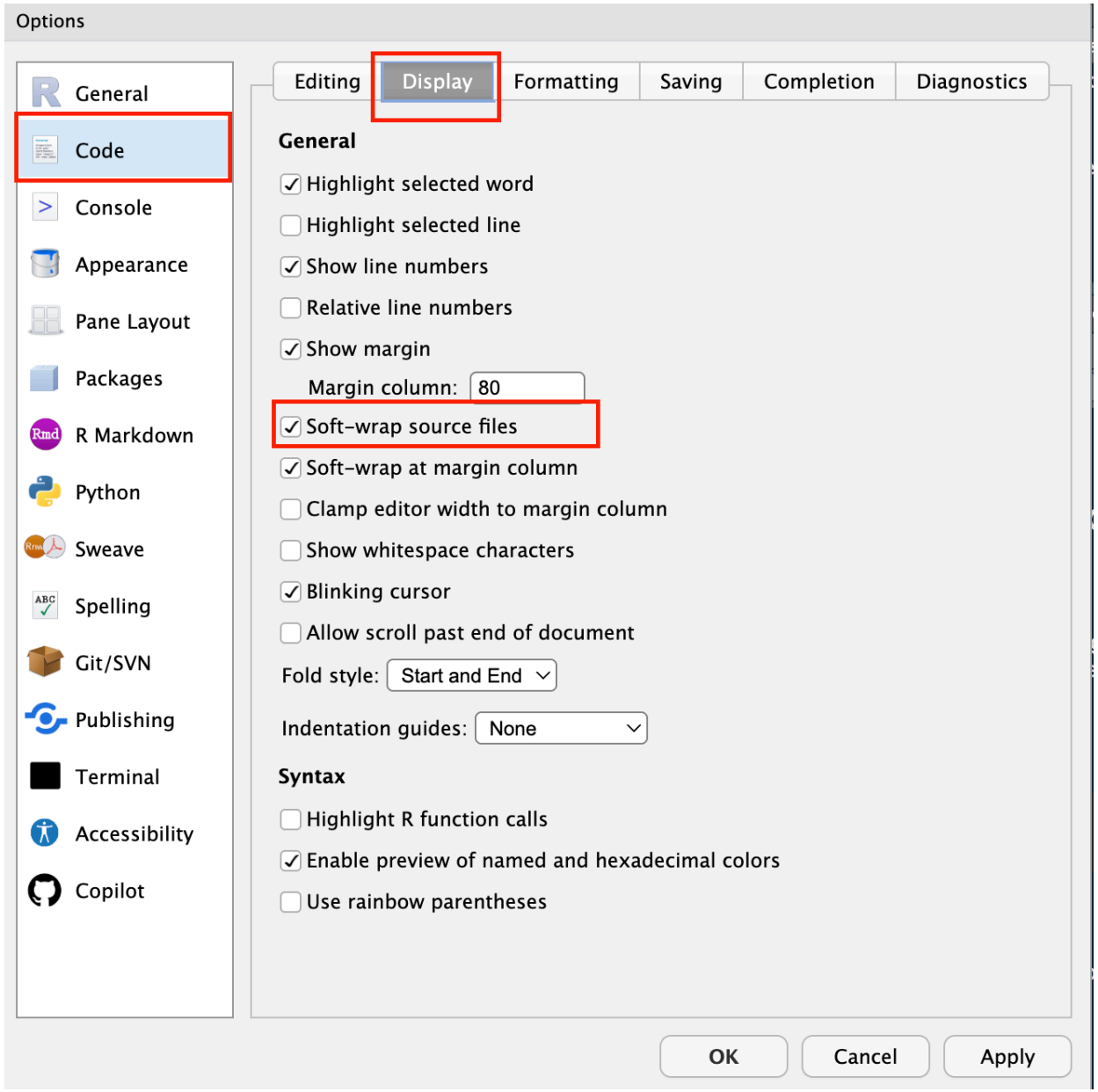
- Restoring the workspace from the last session: better turn it off, because "ghost" objects from old sessions cause errors. On the `General` tab, untick `Restore .RData into workspace at startup` and set `Save workspace to .RData on exit` to `Never`.
- Wrapping long lines in the code editor is convenient: on the `Code` tab, tick `Soft-wrap R source files`.

RStudio settings: General tab



General tab: untick Restore .RData, set Save workspace to Never.

RStudio settings: Code tab



Code tab: tick Soft-wrap R source files.

Installing packages

A **package** adds new functions to R. Packages are downloaded from **CRAN**; choose a nearby CRAN mirror so that downloads are fast.

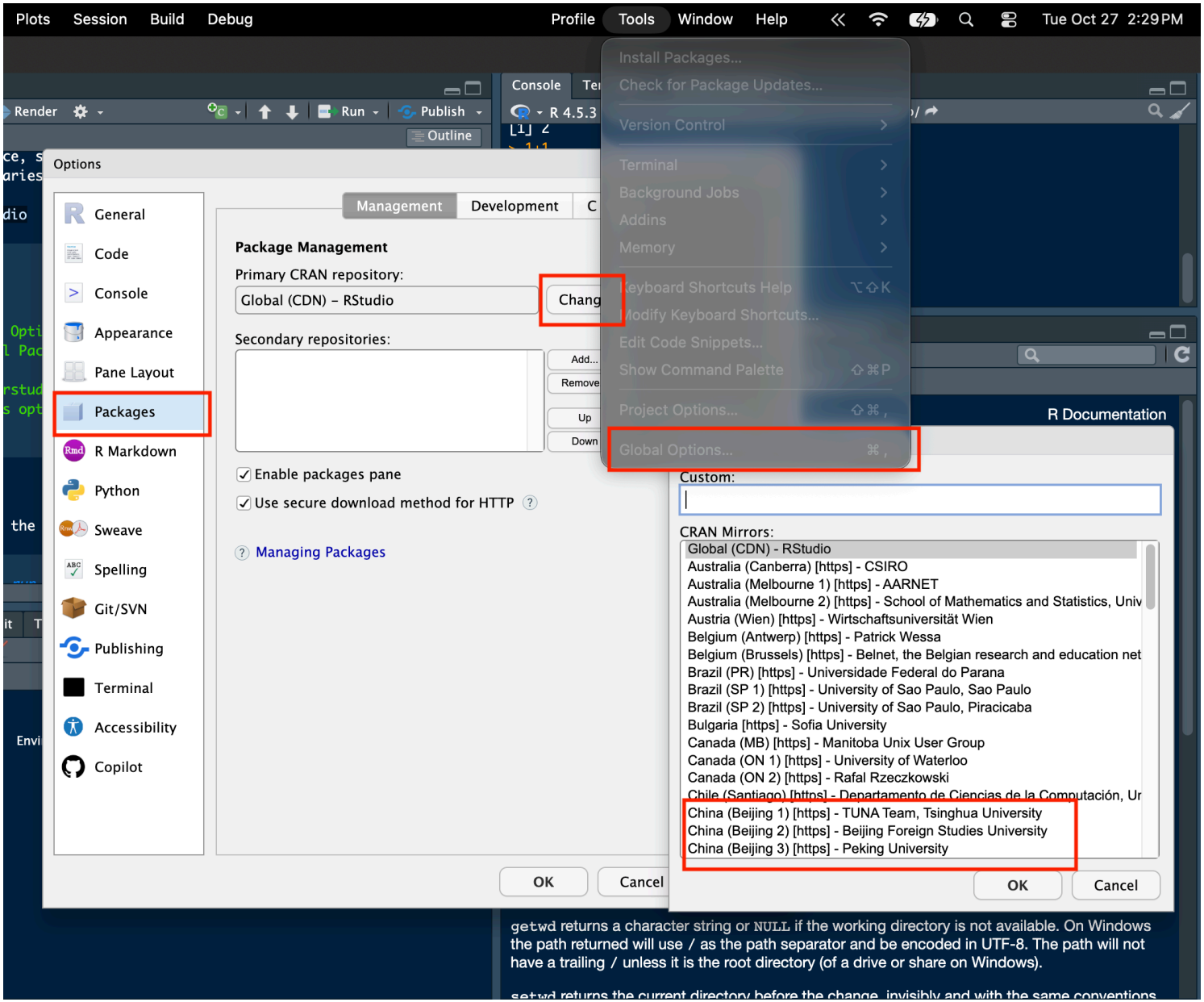
- `Tools -> Global Options -> Packages -> Primary CRAN repository`: choose a mirror in China (e.g. Tsinghua).
- Or run once per session: `options(repos = c(CRAN = "https://mirrors.tuna.tsinghua.edu.cn/CRAN/"))`

Install a package once (`install.packages()` or the `Install` button in the Packages pane); load it every time you start R with `library()`. More on this below, with tibbles.

On Windows, CRAN gives ready-to-use packages. They are saved in your personal library, e.g. `C:\Users\<your name>\AppData\Local\R\win-library\<R version>`; `.libPaths()` shows where.

macOS / Linux: macOS also gets ready-to-use packages; on Linux, packages are compiled from source, so installation is slower and some packages need extra system libraries.

Installing packages: in RStudio



Packages: click Change… and choose a CRAN mirror in China.

Comments

Comments in code start with the # symbol.

```
# this is a comment, R will not run it
```

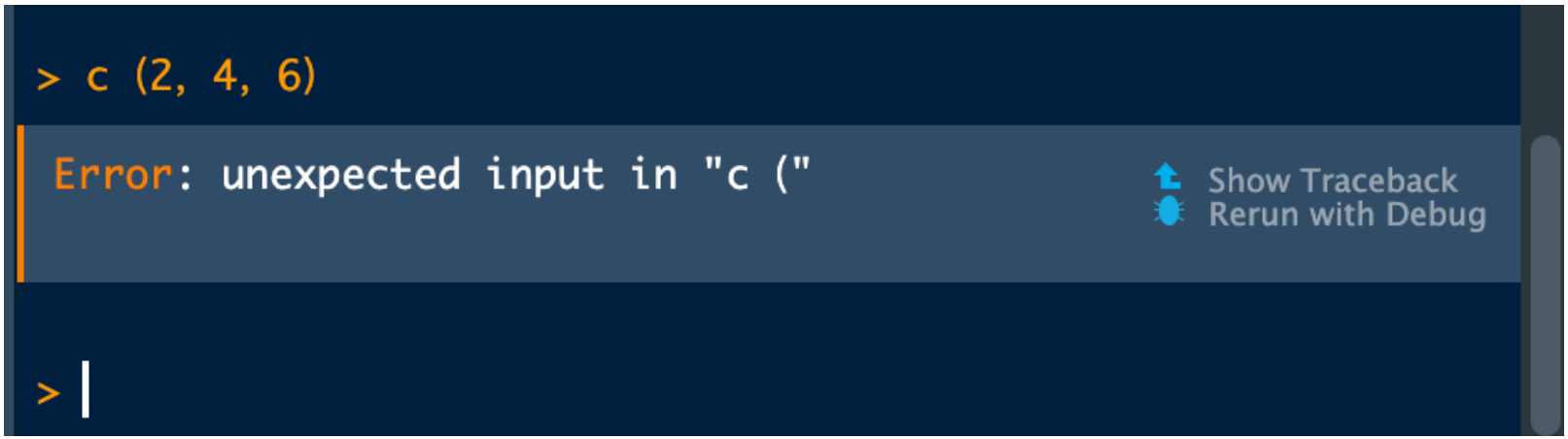
Useful RStudio keyboard shortcuts

- `Ctrl + Shift + N`: new script; `Ctrl + S`: save the script.
- `Ctrl + Enter`: send the current line from the script to the console; if several lines are selected, all of them are run.
- `Ctrl + Shift + Enter`: run the whole script.
- `Ctrl + Shift + C`: comment/uncomment the selected lines.
- `Alt` + drag the mouse: select a rectangular block of text (several lines at once).
- `Tab` or `Ctrl + Space`: press after you start typing a function or variable name to get a list of suggestions. This helps you type faster and with fewer mistakes.
- `Ctrl + L`: clear the console.
- `Alt + -`: type the assignment operator (`<-`).
- `Ctrl + Shift + M`: type the pipe operator `%>%`. We start using it on Nov 2; it needs the `dplyr` package (part of `tidyverse`).

macOS: `Cmd` instead of `Ctrl`, and `Option` instead of `Alt` (but `Ctrl + L` and `Ctrl + Space` stay the same).

Type code with English input

R understands only "half-width" (English) symbols. Brackets, commas and quotation marks typed with a Chinese input method look similar, but they are different characters, and R stops with an error:



Full-width brackets and commas: R stops with unexpected input.

```
c(2, 4, 6) # half-width: works
```

If R reports unexpected `input` in a line that looks correct, retype its brackets, commas and quotation marks in English input.

R as a calculator

R can do arithmetic with **operators** (+, -, *, /, ^) and with functions such as `sqrt()`; `:` makes a sequence of numbers. Operators, functions and vectors are explained in the next slides.

```
2+2
## [1] 4
1024/2
## [1] 512
1:10
##  [1]  1  2  3  4  5  6  7  8  9 10
34*4
## [1] 136
2^4
## [1] 16
sqrt(27)
## [1] 5.196152
```

Variables

The **assignment operator** is the arrow `<-`. It (usually) works from right to left: the value of the expression on the right is stored in the object on the left. Type it with `Alt + -`.

= also works for assignment (`x = 5`), but in this course we use `<-`. = is used inside functions to give arguments, e.g. `seq(from = 1, to = 5)`.

A **variable** is a container: you can put different data and even functions into it. Anything stored in R under a name is an **object**.

Variable names can contain upper- and lower-case Latin letters, dots (`.`), underscores (_) and numbers, and must start with a letter. Choose clear, meaningful names.

```
var_1 <- 1024 / 2
1238 * 3 -> var_2
var_2
## [1] 3714
```

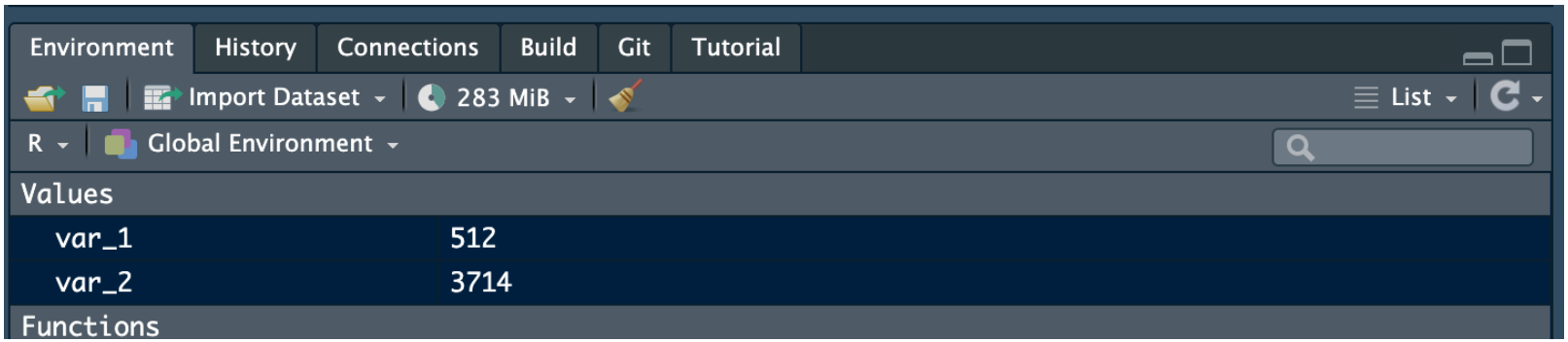
macOS: `Option + -` types `<-`.

How to name a variable?

- `a`: bad, and so are `b`, `c` or `x`. Sometimes acceptable, though :)
- `var1`: bad, but better
- `var_1`: bad, but better
- `seqlength`: meaningful, but hard to read
- `seq_length`, `gc_content` or `read_count`: good, meaningful and easy to read

The Environment pane shows all the objects you have created.

How to name a variable: the Environment pane



The Environment pane lists `var_1` and `var_2` with their values.

Vectors

R can store data in different kinds of objects. R is a vector language: most objects in R are vectors.

There are two kinds of vectors:

- atomic vectors: all elements have the same data type;
- lists: elements can have different types.

Examples of atomic vectors:

```
23
## [1] 23
sqrt(25)
## [1] 5
```

These are vectors of length 1: a single value is also a vector.

A **vector** is one object that holds one or more values, all of the same type.

Some ways to create vectors

- The `:` operator creates integer vectors with consecutive values.

```
1:10 # from 1 to 10
##  [1]  1  2  3  4  5  6  7  8  9 10
-5:3 # from -5 to 3
## [1] -5 -4 -3 -2 -1  0  1  2  3
```

- The **function** `seq()` creates sequences of numbers.

```
seq(from = 1, to = 5, by = 0.5)
## [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0
```

`seq()` is a function, and `from`, `to` and `by` are its **arguments.** Functions and arguments are explained in more detail later.

Some ways to create vectors

- The function c ( ) (from "concatenate").

```
?c # read the help page for this function
```

The function c ( ) takes any number of arguments separated by commas and puts them together into a vector.

```
c(2, 4, 6)
## [1] 2 4 6
c(-9.3, 0, 2.17, 21.3)
## [1] -9.30  0.00  2.17 21.30
```

Vectors can be stored in variables to use later

```
vect_num <- -11:12 # numeric vector from -11 to 12, stored in vect_num
```

Indexing vectors

With the [ ] operator, you can get some elements of a vector. Inside the square brackets, give the position numbers (the **index**) of one or several elements.

```
vect_num[1] # the first element of vect_num
## [1] -11
vect_num[10] # the 10th element
## [1] -2
vect_num[22]
## [1] 10
```

Indexing vectors

To get several elements, give their positions as a vector. For example, to get elements 3 to 5, we need a vector that contains 3, 4 and 5:

```
3:5
## [1] 3 4 5
```

If we put it in square brackets, we get the elements at these positions

```
vect_num[3:5]
## [1] -9 -8 -7
```

Selecting elements from a vector

In the same way, if the elements you need are not next to each other, give a vector of their positions made with `c()`. `c(2, 4, 6)` is a vector containing 2, 4 and 6, so

```
vect_num[c(2, 4, 6)] # returns the 2nd, 4th and 6th elements
## [1] -10  -8  -6
vect_num[c(1, 10, 20)] # returns the 1st, 10th and 20th elements
## [1] -11  -2   8
```

Why does this give an error?

We want the 1st, 3rd and 5th elements, so we list them inside the square brackets:

```
vect_num[1, 3, 5]
```

Why does this give an error?

Why does this give an error? Answer

```
vect_num[1, 3, 5]
## Error in `vect_num[1, 3, 5]`:
## ! incorrect number of dimensions
```

Inside [ ], commas separate dimensions: rows and columns of a table (coming later). A vector has only one dimension, so its positions must be combined into one vector with c( ):

```
vect_num[c(1, 3, 5)] # correct
## [1] -11  -9  -7
```

Combining vectors

The function c( ) can combine several vectors into one

```
c(1, 1, 5:9)
## [1] 1 1 5 6 7 8 9
c(vect_num, vect_num)
##  [1] -11 -10  -9  -8  -7  -6  -5  -4  -3  -2  -1   0   1   2   3   4
## [17]   5   6   7   8   9  10  11  12 -11 -10  -9  -8  -7  -6  -5  -4
## [33]  -3  -2  -1   0   1   2   3   4   5   6   7   8   9  10  11  12
c(100, vect_num)
##  [1] 100 -11 -10  -9  -8  -7  -6  -5  -4  -3  -2  -1   0   1   2   3
## [17]   4   5   6   7   8   9  10  11  12
```

Getting elements 1, 3, 5 and 22 to 24

```
vect_num[c(1, 3, 5, 22:24)]
## [1] -11  -9  -7  10  11  12
```

Your turn 1: vectors

The lengths (bp) of 8 sequencing reads:

```
read_lengths <- c(150, 148, 151, 90, 150, 149, 75, 150)
```

1. Get the 2nd and the 4th read lengths.
2. Get the last element, using `length()` instead of typing 8.
3. Make a vector `all_reads` that combines `read_lengths` with two more reads, 120 and 151 bp. How many elements does it have?

Your turn 1: answers

```
read_lengths[c(2, 4)]
## [1] 148  90
read_lengths[length(read_lengths)]
## [1] 150
all_reads <- c(read_lengths, 120, 151)
length(all_reads)
## [1] 10
```

Data types in R

Numeric data

The **data type** of everything you saw in the previous section is **numeric**. R stores numbers in two ways:

- **integer**: whole numbers, e.g. the numbers made by `:`;
- **double**: numbers that can have a decimal part. Every number you type, even `5`, is stored as a double.

The function `class()` tells you what kind of data an object holds:

```
class(1:10)
## [1] "integer"
class(2.17)
## [1] "numeric"
class(5)
## [1] "numeric"
```

`class()` shows doubles simply as `"numeric"`. For calculations the difference almost never matters.

Character data

Each text element (a "string", or **character** value) must be surrounded by quotation marks. Double and single quotes work the same; use one kind inside the other when the text contains quotes, e.g. `"it's"` or `'the "A" allele'`.

```
"this is text"
## [1] "this is text"
'this is text, too'
## [1] "this is text, too"
```

Text values can be combined into a character vector.

```
dna_seq <- c("A", "T", "G", "C", "T", "A") # a short DNA fragment
dna_seq # the whole vector
## [1] "A" "T" "G" "C" "T" "A"
```

Selecting from a character vector

Getting the first and the last elements. We know that here there are 6 elements.

```
dna_seq[c(1, 6)]
## [1] "A" "A"
```

Getting elements 5 to 10. dna_seq has only 6 elements, so first we glue a longer vector from two copies of it.

```
double_seq <- c(dna_seq, dna_seq)
double_seq
##  [1] "A" "T" "G" "C" "T" "A" "A" "T" "G" "C" "T" "A"
double_seq[5:10] # elements 5 to 10
## [1] "T" "A" "A" "T" "G" "C"
```

Logical data

```
TRUE # true
## [1] TRUE
FALSE # false
## [1] FALSE
```

Logical values can be shortened to their first capital letters. **But better don't do this**: the code is easier to read in full.

```
c(T, T, T, T, F, F, T, T)
## [1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE  TRUE  TRUE
```

Creating a logical vector.

```
c(TRUE, TRUE, TRUE, FALSE, FALSE, TRUE)
## [1]  TRUE  TRUE  TRUE FALSE FALSE  TRUE
```

One more logical vector

```
short_logical_vector <- c(FALSE, TRUE)
```

A vector holds one data type

If you mix types in one vector, R converts all values to one type, in the order logical -> numeric -> character:

```
c(1, "A") # a number and text: everything becomes text
## [1] "1" "A"
class(c(1, "A"))
## [1] "character"
c(1, TRUE) # a number and a logical: TRUE becomes 1
## [1] 1 1
class(c(1, TRUE))
## [1] "numeric"
```

Long vectors

To create long vectors of repeated elements, use the function `rep()`.

```
rep(x = 1, times = 3) # 1 is repeated 3 times
## [1] 1 1 1
rep(x = "A", times = 5) # "A" is repeated 5 times (a poly-A tail!)
## [1] "A" "A" "A" "A" "A"
rep(x = TRUE, times = 2) # TRUE is repeated 2 times
## [1] TRUE TRUE
```

What is a function?

A function is a named command that does something. You call it by its name with round brackets: `name(...)`.

The arguments are the inputs inside the brackets. Each is written `name = value`, and arguments are separated by commas. Some arguments have default values, so you can leave them out.

```
rep(x = "A", times = 5) # function rep(), arguments x and times
## [1] "A" "A" "A" "A" "A"
```

Function arguments

In R, you can leave out the names of function arguments if you give them in the order listed in the help page of the function.

```
rep(TRUE, 5) # TRUE repeated 5 times, arguments without names
## [1] TRUE TRUE TRUE TRUE TRUE
```

Let's create a logical vector where TRUE is repeated 3 times, FALSE 3 times and TRUE 4 times, and store it in the variable `vect_log`.

```
vect_log <- c(rep(TRUE, 3), rep(FALSE, 3), rep(TRUE, 4))
vect_log
##  [1]  TRUE  TRUE  TRUE FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE
```

Logical operators

Logical operators compare values or combine conditions. The result is a logical vector: TRUE or FALSE for each element.

Operator	Meaning	Example
`>`	greater than	`vect_num > 0`
`<`	less than	`vect_num < -5`
`>=`	greater than or equal to	`vect_num >= 8`
`<=`	less than or equal to	`vect_num <= -8`
`==`	equal to	`double_seq == "G"`
`!=`	not equal to	`double_seq != "A"`
`!`	not: turns TRUE into FALSE and back	`!(double_seq == "G")`
`&`	and: both conditions are TRUE	`vect_num > 0 & vect_num < 5`
`|`	or: at least one condition is TRUE	`vect_num < -10 | vect_num > 10`

Using logical vectors to filter data

Logical vectors made with logical operators can be used for **filtering** data.

Remember our character vector:

```
double_seq
##  [1] "A" "T" "G" "C" "T" "A" "A" "T" "G" "C" "T" "A"
```

Example 1

Getting only the guanines (`"G"`).

We can create a logical vector that is TRUE only for the 3rd and 9th elements.

```
f_g <- double_seq == "G"
f_g
##  [1] FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE
## [12] FALSE
```

We can use this logical vector as a filter to get data from `double_seq`

```
double_seq[f_g]
## [1] "G" "G"
```

Example 2

Getting all G and C from `double_seq` (to count the GC content). We already have the G filter, so we create a filter for C.

```
f_c <- double_seq == "C"
```

"G or C" can be written with the logical "or" (`|`).

```
f_g | f_c
##  [1] FALSE FALSE  TRUE  TRUE FALSE FALSE FALSE FALSE  TRUE  TRUE FALSE
## [12] FALSE
```

Done: we have extracted all G and C.

```
double_seq[f_g | f_c]
## [1] "G" "C" "G" "C"
```

A shorter way to write it

The same thing can be written in a shorter way.

In one line, very hard to read:

```
double_seq[double_seq == "G" | double_seq == "C"]
## [1] "G" "C" "G" "C"
```

With a separate filter, easier to read:

```
f_gc <- double_seq == "G" | double_seq == "C"
double_seq[f_gc]
## [1] "G" "C" "G" "C"
```

The same filter in the tidyverse

The **tidyverse** is a collection of packages for data work, including `tibble`, `dplyr` and `ggplot2`. A preview of the same G/C filter with `filter()` from `dplyr`, on a data frame (a table, explained later):

```
library(dplyr) # part of the tidyverse
filter(data.frame(base = double_seq), base %in% c("G", "C"))
##   base
## 1    G
## 2    C
## 3    G
## 4    C
```

We will use the tidyverse from Nov 2. The book uses the pipe `|>`; in this course we use `%>%`, which works the same way.

Wickham H., Çetinkaya-Rundel M., Grolemund G. (2023). *R for Data Science*, 2nd ed. O'Reilly. Free online: https://r4ds.hadley.nz.

GC content from a logical vector

In calculations, TRUE counts as `1` and FALSE as 0. So `sum()` of a logical vector counts the TRUE values, and `mean()` gives their proportion.

```
f_gc
##  [1] FALSE FALSE  TRUE  TRUE FALSE FALSE FALSE FALSE  TRUE  TRUE FALSE
## [12] FALSE
sum(f_gc) # number of G and C bases
## [1] 4
mean(f_gc) # GC content: the proportion of G and C, 4 of 12 bases
## [1] 0.3333333
```

The operator `%in%` checks whether each value is one of several values: a shorter way to write the same filter.

```
f_gc2 <- double_seq %in% c("G", "C")
mean(f_gc2)
## [1] 0.3333333
```

Example 3

We had a numeric vector.

```
vect_num
##  [1] -11 -10  -9  -8  -7  -6  -5  -4  -3  -2  -1   0   1   2   3   4
## [17]   5   6   7   8   9  10  11  12
```

Getting only the values greater than 0 from `vect_num`.

```
vect_num[vect_num > 0]
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12
```

Example 4

Getting from `vect_num` all numbers that are either less than or equal to -8, or greater than or equal to 8.

```
f_min8_plus8 <- (vect_num <= -8) | (vect_num >= 8)
vect_num[f_min8_plus8]
## [1] -11 -10  -9  -8   8   9  10  11  12
```

Your turn 2: logical filtering

Use the vector `read_lengths` from "Your turn 1".

1. Get the reads shorter than 100 bp.
2. How many reads are exactly 150 bp long? (Hint: TRUE counts as 1.)
3. What proportion of the reads are at least 148 bp long?

Your turn 2: answers

```
read_lengths[read_lengths < 100]
## [1] 90 75
sum(read_lengths == 150)
## [1] 3
mean(read_lengths >= 148)
## [1] 0.75
```

Factors

A **factor** is the R object for discrete (= **categorical**) data. A factor defines the categories of a variable as a fixed set of levels: R knows which values are allowed and in which order the groups come. Factors are built on integer vectors and have extra attributes (`class` and `levels`).

For example, you genotyped 8 individuals at one SNP (a single-nucleotide polymorphism) with the alleles A and G. There are many individuals, but only three possible genotypes.

```
snp_genotypes <- c("AG", "AA", "AA", "AA", "GG", "GG", "GG", "GG")
snp_genotypes # this is a character vector
## [1] "AG" "AA" "AA" "AA" "GG" "GG" "GG" "GG"
```

"GG" means the same thing for every individual: genotype is a variable with three possible categories. A factor stores each value as a number matched with a label (called a **level**): the genotype name. Later in the course, statistical models (ANOVA, GLM) use factors to compare groups, so categorical predictors must be factors. We can create the factor "genotype".

Creating a factor

```
factor(snp_genotypes)
## [1] AG AA AA AA GG GG GG GG
## Levels: AA AG GG
```

The levels of this factor are

- 1: AA,
- 2: AG,
- 3: GG.

By default, R sorts the levels alphabetically. You can change the order (see `help("factor")`). We will need this later.

```
double_seq # character vector
##  [1] "A" "T" "G" "C" "T" "A" "A" "T" "G" "C" "T" "A"
```

Creating a factor from a character vector and storing it in a variable

```
f_double_seq <- factor(double_seq)
```

What kind of data is stored in a variable?

To find out what kind of data is stored in a variable, use the function `class()`

```
class(f_double_seq)
## [1] "factor"
class(vect_log)
## [1] "logical"
class(vect_num)
## [1] "integer"
class(dna_seq)
## [1] "character"
```

Built-in constants in R

Built-in constants in R: `NA`, `NULL`, `NaN`, `Inf`

- **NA**: "not available". The object exists, but its property was not measured or recorded.
- **NULL**: empty, there is simply nothing.
- **NaN**: "not a number".
- **Inf**: "infinity".

```
dna_seq_1 <- c("A", "T", NA, "C", "T", "A") # NA: 3rd base not read
length(NULL) # NULL: nothing, so no elements
## [1] 0
0 / 0 # NaN
## [1] NaN
1 / 0 # Inf
## [1] Inf
```

An element that does not exist

What happens if you ask for an element number that does not exist?

An element that does not exist

What happens if you ask for an element number that does not exist?

R returns NA, because there is no such element.

```
dna_seq_1[198]
## [1] NA
```

Arithmetic with vectors

Operations work element by element: `vect_num + 2` adds 2 to every element.

```
vect_num
##  [1] -11 -10  -9  -8  -7  -6  -5  -4  -3  -2  -1   0   1   2   3   4
## [17]   5   6   7   8   9  10  11  12

vect_num + 2
##  [1] -9 -8 -7 -6 -5 -4 -3 -2 -1  0  1  2  3  4  5  6  7  8  9 10 11 12
## [23] 13 14
vect_num * 2
##  [1] -22 -20 -18 -16 -14 -12 -10  -8  -6  -4  -2   0   2   4   6   8
## [17]  10  12  14  16  18  20  22  24
```

Arithmetic with vectors

```
vect_num * (-2)
##  [1]  22  20  18  16  14  12  10   8   6   4   2   0  -2  -4  -6  -8
## [17] -10 -12 -14 -16 -18 -20 -22 -24
vect_num^2
##  [1] 121 100  81  64  49  36  25  16   9   4   1   0   1   4   9  16
## [17]  25  36  49  64  81 100 121 144
```

Two vectors of the same length are combined position by position:

```
c(1, 2, 3) + c(10, 20, 30)
## [1] 11 22 33
```

Operations on vectors with constants

```
counts_na <- c(1, 3, NA, 7, 0, 22:24)
```

What happens to NA?

```
counts_na + 2 # NA stays NA
## [1]  3  5 NA  9  2 24 25 26
counts_na * 0 # NA stays NA
## [1]  0  0 NA  0  0  0  0  0
counts_na / 0  # NA stays NA
## [1] Inf Inf  NA Inf NaN Inf Inf Inf
```

But in the last case you will also see:

- Inf when a number is divided by zero;
- NaN when zero is divided by zero.

You also get NaN if you take the square root of a negative number.

```
sqrt(-1)
## [1] NaN
```

Functions in R

You have already seen many functions: you can recognise them by the brackets after the name. Let's meet a few more and learn to write our own functions. Your own functions let you automate repeated steps and make code easy to read.

Here is our vector

```
counts_na
## [1]  1  3 NA  7  0 22 23 24
```

The length of a vector is given by the function `length()`

```
length(counts_na)
## [1] 8
```

The sum of its elements, with the function `sum()`

```
sum(counts_na)
## [1] NA
```

Oops! For some reason we got `NA`.

Functions and vectors with constants

The function `sum()` has the argument `na.rm`, which is FALSE by default: NAs are not removed before the sum is calculated, so the result is NA.

If we give `sum()` the argument `na.rm = TRUE`, we get the correct sum

```
sum(counts_na, na.rm = TRUE)
## [1] 80
```

The same is true for `mean()`, and for `max()` and `min()` (the largest and the smallest value)

```
mean(counts_na, na.rm = TRUE)
## [1] 11.42857
max(counts_na, na.rm = TRUE) # the largest value
## [1] 24
min(counts_na, na.rm = TRUE) # the smallest value
## [1] 0
```

User-defined functions

Let's write our own function `mmean()` that calculates the mean of a vector and ignores missing values (`NA`) by default. This is a **user-defined function**.

```
mmean <- function(x){
  mean(x, na.rm = TRUE)
  }
```

In this code:

- `mmean` is a variable, the name of the function.
- We store in it the function created by `function()`, a function that makes functions. The brackets list the arguments (names of the objects we give to the function to work with).
- `{ }`: the curly brackets contain the body of the function: the steps to perform with the arguments.

Return in the function body

Larger functions often also have the instruction `return()`, which says what exactly the function should return. This is how our function looks with it.

```
mmean <- function(x){
  res <- mean(x, na.rm = TRUE)
  return(res)
}
```

Let's check our function on the vector with NA, against the built-in one

```
mean(counts_na) # without na.rm: NA
## [1] NA
mean(counts_na, na.rm = TRUE)
## [1] 11.42857
mmean(counts_na) # our function ignores NA by default
## [1] 11.42857
```

Matrices

A **matrix** is a two-dimensional object, similar to an atomic vector but with the extra attribute `dim`.

For example, a small count matrix: how many sequencing reads came from each of 3 genes (rows) in each of 3 samples (columns). R fills the matrix column by column.

```
counts <- matrix(c(1520, 980, 12, 1385, 1043, 30, 1610, 875, 7),
                 nrow = 3, ncol = 3)
rownames(counts) <- c("ACTB", "GAPDH", "MYC")
colnames(counts) <- c("sample_1", "sample_2", "sample_3")
counts
##       sample_1 sample_2 sample_3
## ACTB      1520     1385     1610
## GAPDH      980     1043      875
## MYC         12       30        7
```

The dimensions of a matrix are set by two arguments: `nrow` (number of rows) and `ncol` (number of columns).

Matrices: row and column names

`rownames()` and `colnames()` give names to the rows and columns of a matrix:

```
rownames(counts)
## [1] "ACTB"  "GAPDH" "MYC"
colnames(counts)
## [1] "sample_1" "sample_2" "sample_3"
```

The row names are real human gene symbols. ACTB (beta-actin) and GAPDH are **housekeeping genes**: they are needed in almost every cell and have high, stable expression. MYC is a regulatory gene, usually expressed at lower levels. The counts themselves are invented.

Lists

A **list** is a data format where each element can be of any type and any length.

```
my_list <- list(
  12:16,
  snp_genotypes,
  c(TRUE, TRUE, TRUE, FALSE, FALSE, TRUE)
)

str(my_list)
## List of 3
##  $ : int [1:5] 12 13 14 15 16
##  $ : chr [1:8] "AG" "AA" "AA" "AA" ...
##  $ : logi [1:6] TRUE TRUE TRUE FALSE FALSE TRUE
```

Data frames: two-dimensional data structures

A **data frame** is one of the ways to store tables in R. It is built on lists and also has some properties of matrices. It has the attributes `rownames` (row names) and `colnames` (column names). The `names` of a data frame are the same as its `colnames`.

Let's create a data frame. First, we create vectors with data for the variables.

```
ids <- c("gene01", "gene02", "gene03", "gene04", "gene05",
         "gene06", "gene07", "gene08", "gene09") # character: gene IDs
exons <- c(9, 5, 1, 14, 8, 4, 3, 21, 11) # numeric: number of exons
strand <- c("+", "-", "-", "+", "+", "-", "+", "-", "-") # character: strand
# numeric: gene length, bp
len_bp <- c(5230, 18740, 960, 9410, 61200, 2870, 13350, 27600, 7980)
```

Creating a data frame

Now we put these vectors into a data frame

```
my_genes <- data.frame(gene_id = ids, n_exons = exons, length_bp = len_bp,
                       strand = strand)
```

We can check that we really created an object of class data.frame

```
class(my_genes) # check that it is really a data frame
## [1] "data.frame"
```

Converting other objects to a data frame

For example, from a matrix...

```
counts_data <- as.data.frame(counts)
str(counts_data)
## 'data.frame':    3 obs. of  3 variables:
##  $ sample_1: num  1520 980 12
##  $ sample_2: num  1385 1043 30
##  $ sample_3: num  1610 875 7
```

Looking at a data frame

There are several ways to look at the contents of a data frame.

```
my_genes # print the data frame
head(my_genes)
tail(my_genes)
```

```
View(my_genes) # spreadsheet view in RStudio
# fix(my_genes) # manual editing: avoid it, it leaves no record
```

You can also get the number of rows and columns of a data frame.

```
nrow(my_genes)
## [1] 9
ncol(my_genes)
## [1] 4
length(my_genes) # gives the number of columns
## [1] 4
```

Indexing in data frames

Getting a column (variable) with the $ operator and the variable name

```
my_genes$n_exons
## [1]  9  5  1 14  8  4  3 21 11
my_genes$length_bp
## [1]  5230 18740   960  9410 61200  2870 13350 27600  7980
```

Indexing in data frames

Each cell in a data frame has coordinates `[row, column]`

```
my_genes[2, 3] # 2nd row, 3rd column
## [1] 18740
my_genes[2, ] # the whole 2nd row
##   gene_id n_exons length_bp strand
## 2  gene02       5     18740      -
my_genes[1:9, 2] # rows 1 to 9 of the 2nd column
## [1]  9  5  1 14  8  4  3 21 11
my_genes[, 2] # the whole 2nd column
## [1]  9  5  1 14  8  4  3 21 11
```

Expanding a data frame

Adding a column.

```
my_genes$chrom <- c("chr1", "chr1", "chr2", "chr3", "chr2",
                    "chr3", "chrX", "chr2", "chr3")
my_genes
##   gene_id n_exons length_bp strand chrom
## 1  gene01       9      5230      +  chr1
## 2  gene02       5     18740      -  chr1
## 3  gene03       1       960      -  chr2
## 4  gene04      14      9410      +  chr3
## 5  gene05       8     61200      +  chr2
## 6  gene06       4      2870      -  chr3
## 7  gene07       3     13350      +  chrX
## 8  gene08      21     27600      -  chr2
## 9  gene09      11      7980      -  chr3
```

Adding a row

```
my_genes_new <- data.frame(gene_id = "gene10", n_exons = 5,
                           length_bp = 1840, strand = "+", chrom = "chr1")
my_genes_final <- rbind(my_genes, my_genes_new)
my_genes_final
##    gene_id n_exons length_bp strand chrom
## 1   gene01       9      5230      +  chr1
## 2   gene02       5     18740      -  chr1
## 3   gene03       1       960      -  chr2
## 4   gene04      14      9410      +  chr3
## 5   gene05       8     61200      +  chr2
## 6   gene06       4      2870      -  chr3
## 7   gene07       3     13350      +  chrX
## 8   gene08      21     27600      -  chr2
## 9   gene09      11      7980      -  chr3
## 10  gene10       5      1840      +  chr1
```

Your turn 3: selecting from a data frame

Use `my_genes` (with the `chrom` column).

1. Get the strand of the 5th gene in two ways: with `$` and with `[row, column]`.
2. Get the IDs of all genes on the minus strand (`"-"`).
3. Get rows 2 to 4 of the columns `gene_id` and `length_bp` (columns 1 and 3).

Your turn 3: answers

```
my_genes$strand[5]
## [1] "+"
my_genes[5, 4]
## [1] "+"
my_genes$gene_id[my_genes$strand == "-"]
## [1] "gene02" "gene03" "gene06" "gene08" "gene09"
my_genes[2:4, c(1, 3)]
##   gene_id length_bp
## 2  gene02     18740
## 3  gene03       960
## 4  gene04      9410
```

Tibbles

A **tibble** is a slightly modified data frame with a similar structure; it is a newer addition to R. Unlike data frames, tibbles have the extra class `tbl_df`, which makes them behave a little differently. Tibbles come from the package `tibble`, part of the tidyverse.

You install a package into your local library only once. So the `install.packages()` line should not stay in the final version of your script.

```
# install.packages("tidyverse")
```

In each R session, you must load a package before you use it. Next time you start R, you will have to load the packages again. So the lines that load packages with `library()` must stay in the final version of your script.

```
library(tibble)
my_tibble <- tibble(gene_id = ids, n_exons = exons,
                    length_bp = len_bp, strand = strand)
str(my_tibble)
## tibble [9 x 4] (S3: tbl_df/tbl/data.frame)
##  $ gene_id  : chr [1:9] "gene01" "gene02" "gene03" "gene04" ...
##  $ n_exons  : num [1:9] 9 5 1 14 8 4 3 21 11
##  $ length_bp: num [1:9] 5230 18740 960 9410 61200 ...
##  $ strand   : chr [1:9] "+" "-" "-" "+" ...
```

Tibbles

One difference between tibbles and data frames is how they are created. When you create a tibble, you can already use a column you have just created. Here `length_kb` is calculated from `length_bp` (`data.frame()` cannot do this):

```
my_tibble2 <- tibble(gene_id = ids, length_bp = len_bp,
                     length_kb = length_bp / 1000, strand = strand)
my_tibble2
## # A tibble: 9 x 4
##   gene_id length_bp length_kb strand
##   <chr>       <dbl>     <dbl> <chr>
## 1 gene01       5230      5.23 +
## 2 gene02      18740     18.7  -
## 3 gene03        960      0.96 -
## 4 gene04       9410      9.41 +
## 5 gene05      61200     61.2  +
## 6 gene06       2870      2.87 -
## 7 gene07      13350     13.4  +
## 8 gene08      27600     27.6  -
## 9 gene09       7980      7.98 -
```

The tidyverse packages (e.g. `dplyr`, `ggplot2`) are built to work with tibbles. Tibbles also respond differently to some functions.

Reading and writing data

Reading a CSV file

A **CSV file** is a plain-text table: one row per line, values separated by commas. Save `genes.csv` (made-up practice data, from Moodle) into the `data` folder of your course project, then read it:

```
genes <- read.csv("data/genes.csv")
head(genes)
##   gene_id chrom strand n_exons length_bp gc_content
## 1  gene01  chr1      +       9      5230       0.48
## 2  gene02  chr1      -       5     18740       0.44
## 3  gene03  chr2      -       1       960       0.57
## 4  gene04  chr3      +      14      9410       0.51
## 5  gene05  chr2      +       8     61200       0.39
## 6  gene06  chr3      -       4      2870       0.62
```

`"data/genes.csv"` is a relative path: R looks for the file inside the working directory, i.e. your project folder. If R says `cannot open file ... No such file or directory`, check that you opened the project and that the file is in `data`.

Checking what you have read

Always check the structure of a table you have just read:

```
str(genes)
## 'data.frame':    15 obs. of  6 variables:
##  $ gene_id   : chr  "gene01" "gene02" "gene03" "gene04" ...
##  $ chrom     : chr  "chr1" "chr1" "chr2" "chr3" ...
##  $ strand    : chr  "+" "-" "-" "+" ...
##  $ n_exons   : int  9 5 1 14 8 4 3 21 11 5 ...
##  $ length_bp : int  5230 18740 960 9410 61200 2870 13350 27600 7980 1840
 ↪  ...
##  $ gc_content: num  0.48 0.44 0.57 0.51 0.39 0.62 0.42 0.46 0.53 0.59 ...
```

`read.csv()` returns a data frame: 15 observations (genes) of 6 variables. Text columns are character, numbers are integer or numeric (double).

Writing a CSV file

Select the genes longer than 20 kb: a logical vector in the row position, and an empty column position (= all columns).

```
long_genes <- genes[genes$length_bp > 20000, ]
long_genes
##    gene_id chrom strand n_exons length_bp gc_content
## 5   gene05  chr2      +       8     61200       0.39
## 8   gene08  chr2      -      21     27600       0.46
## 11  gene11  chr2      +       6     22150       0.41
## 13  gene13  chrX      -      12     38900       0.38
```

Save them into a new file in `data`. `row.names = FALSE` stops R from adding a column with row numbers.

```
write.csv(long_genes, "data/long_genes.csv", row.names = FALSE)
```

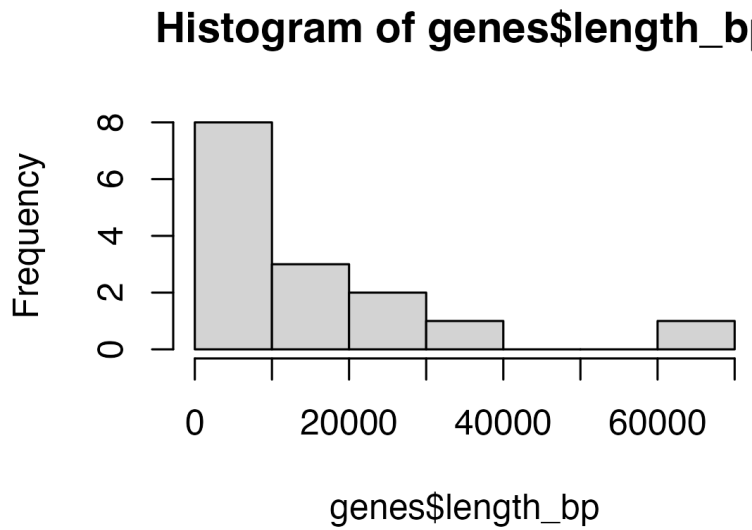
The file appears in `data/` and in the `Files` pane; Excel can open it.

A first look at the data

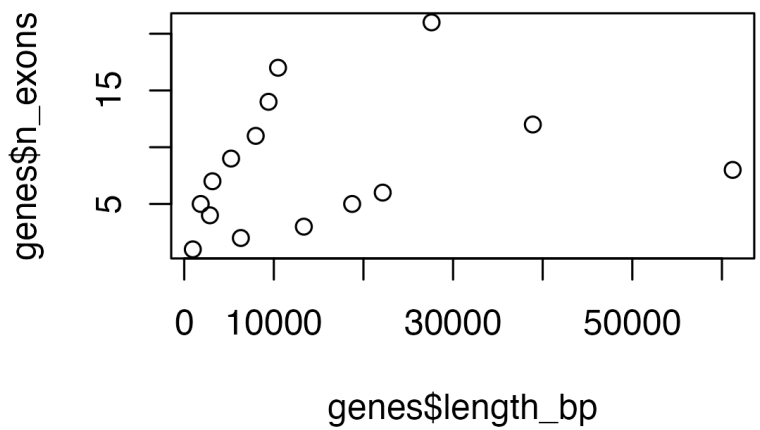
Quick-look plots in base R

Base R can draw simple plots for a quick check of your data. Proper, publication-quality plots with `ggplot2` come in the next session.

```
hist(genes$length_bp)
```




```
plot(x = genes$length_bp,
     y = genes$n_exons)
```



What we have learned

- **R** runs your code; **RStudio** is the IDE where you write it. Keep your code in a **script** and open your course folder as an RStudio **project**, so the **working directory** is set for you. In Windows paths, write `/` instead of `\`.
- Anything stored in R under a name is an **object**. The **assignment operator** `<-` stores a value in a **variable**.
- A **vector** is an ordered set of values of the same **data type**: numeric, character or logical. Select its elements by **index** (`x[3]`) or by **filtering** with a logical vector (`x[x > 0]`); `mean()` of a logical vector is a proportion, e.g. the GC content.
- A **factor** stores a categorical variable with a fixed set of **levels**. **NA** is a missing value: many functions return NA unless you set `na.rm = TRUE`.
- A **data frame** stores a data table: each column is a vector (one variable), each row is one observation. Select from it with `$` and `[row, column]`.
- Read a table with `read.csv()` and save one with `write.csv()`, using **relative paths** inside the project. A **package** adds new functions: install it once, load it with `library()` every time you start R.

Key R objects and functions: objects

Object	What it is	Example
vector	An ordered set of one or more values, all of the same type	`c("A", "T", "G", "C")`
factor	A categorical variable; each value is one of a fixed set of levels	`factor(c("AG", "AA", "GG"))`
matrix	A two-dimensional table of values of one type	`matrix(1:9, nrow = 3, ncol = 3)`
list	Holds elements of different types and lengths	`list(12:16, c(TRUE, FALSE))`
data frame	A data table: each column is a vector (one variable), all columns have the same length	`data.frame(gene_id = c("gene01", "gene02"), strand = c("+", "-"))`
tibble	A modern version of a data frame (package `tibble`, part of the tidyverse)	`tibble(n_exons = 1:3)`

Key R objects and functions: setup and vectors

Function	What it does	Example
`<-`	Stores a value in a variable	`x <- c(2, 4, 6)`
`=`	Gives a value to a function argument	`seq(from = 1, to = 5)`
`help()`, `?`	Opens the help page of a function	`?seq`
`setwd()`	Sets the working directory	`setwd("C:/Users/student/Documents")`
`install.packages()`	Installs a package (once)	`install.packages("tidyverse")`
`library()`	Loads a package (every R session)	`library(tibble)`
`c()`	Combines values into a vector	`c(-9.3, 0, 2.17)`
`:`	Integer sequence with step `1`	`1:10`
`seq()`	Sequence of numbers with any step	`seq(from = 1, to = 5, by = 0.5)`
`rep()`	Repeats values	`rep(x = "A", times = 5)`
`[ ]`	Selects elements by index or by a logical filter	`x[c(1, 3)]`, `x[x > 0]`

Key R objects and functions: working with vectors

Function	What it does	Example
`==, !=, >, <, !, &, |`	Logical operators: compare values, combine conditions (not, and, or)	`x == "G", x > 0 & x < 8`
`%in%`	Is each value one of several values?	`x %in% c("G", "C")`
`filter()`	(`dplyr`, preview) Keeps the rows of a table that meet a condition	`filter(data.frame(base = x), base == "G")`
`factor()`	Turns a vector into a factor	`factor(snp_genotypes)`
`class()`	Shows what kind of object a variable holds	`class(dna_seq)`
`length()`	Number of elements in a vector	`length(x)`
`sum()`	Sum of the elements; `na.rm = TRUE` ignores NA	`sum(x, na.rm = TRUE)`
`mean()`	Arithmetic mean; `na.rm = TRUE` ignores NA; of a logical vector: a proportion	`mean(x, na.rm = TRUE), mean(f_gc)`
`max(), min()`	Largest / smallest value; `na.rm = TRUE` ignores NA	`max(x, na.rm = TRUE)`
`sqrt()`	Square root	`sqrt(27)`
`function()`	Creates a user-defined function	`double_it <- function(x){ x * 2 }`
`return()`	Says what a function returns	`return(res)`

Key R objects and functions: tables and plots

Function	What it does	Example
`matrix()`	Creates a matrix; `nrow`, `ncol` set its size	`matrix(1:9, nrow = 3, ncol = 3)`
`rownames()`, `colnames()`	Names of rows / columns; can also set them	`rownames(counts) <- c("ACTB", "GAPDH", "MYC")`
`list()`	Creates a list	`list(12:16, snp_genotypes)`
`data.frame()`	Creates a data frame from vectors	`data.frame(n_exons = exons, strand = strand)`
`as.data.frame()`	Converts another object to a data frame	`as.data.frame(counts)`
`nrow()`, `ncol()`	Number of rows / columns	`nrow(my_genes)`
`$`, `[row, column]`	Selects a column; selects cells	`my_genes$n_exons`, `my_genes[2, 3]`
`rbind()`	Adds rows to a data frame	`rbind(my_genes, my_genes_new)`
`tibble()`	Creates a tibble	`tibble(n_exons = exons, strand = strand)`

Key R objects and functions: data files and plots

Function	What it does	Example
`read.csv()`	Reads a CSV file into a data frame	`genes <- read.csv("data/genes.csv")`
`write.csv()`	Saves a data frame as a CSV file; `row.names = FALSE` drops the row numbers	`write.csv(long_genes, "data/long_genes.csv", row.names = FALSE)`
`str()`	Shows the structure of an object	`str(genes)`
`head()`, `tail()`, `View()`	First rows, last rows, a spreadsheet view	`head(genes)`
`hist()`, `plot()`	Quick histogram; quick scatterplot	`hist(genes$length_bp)`